MACHINE LEARNING WITH SCIKIT-LEARN

# Classification & Ensembles

06

Open every remaining black box — logistic, trees, SVM — then let committees of models outvote any single one.

**Instructor: Shutchon Premchaisawatt**

Master's Degree Program · Hands-on with Google Colab

AGENDA

# Today's agenda

**01** **Decision boundaries**
what a classifier actually draws

**02** **Logistic regression**
sigmoid, probabilities, weights

**03** **Decision trees**
rules you can read aloud

**04** **Naive Bayes & SVM**
the 90-second tour

**05** **Why ensembles**
one tree = opinion, many = wisdom

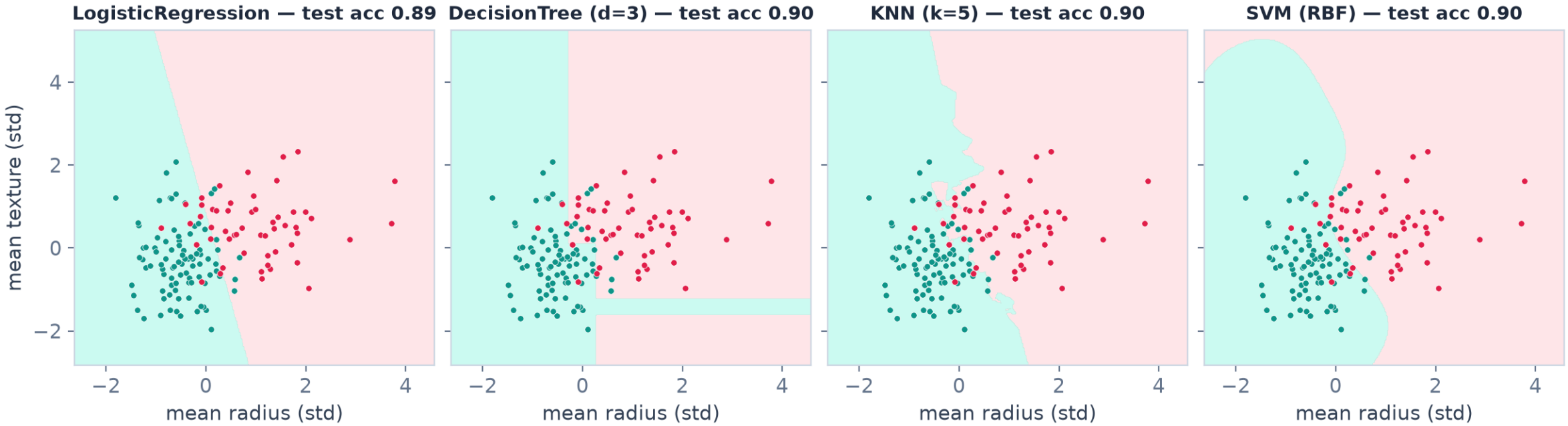
**06** **Bagging → RandomForest**
+ feature importances

**07** **Boosting → HistGradientBoosting**
and XGBoost beyond

**08** **Voting & the full scoreboard**
who wins when

CONCEPT

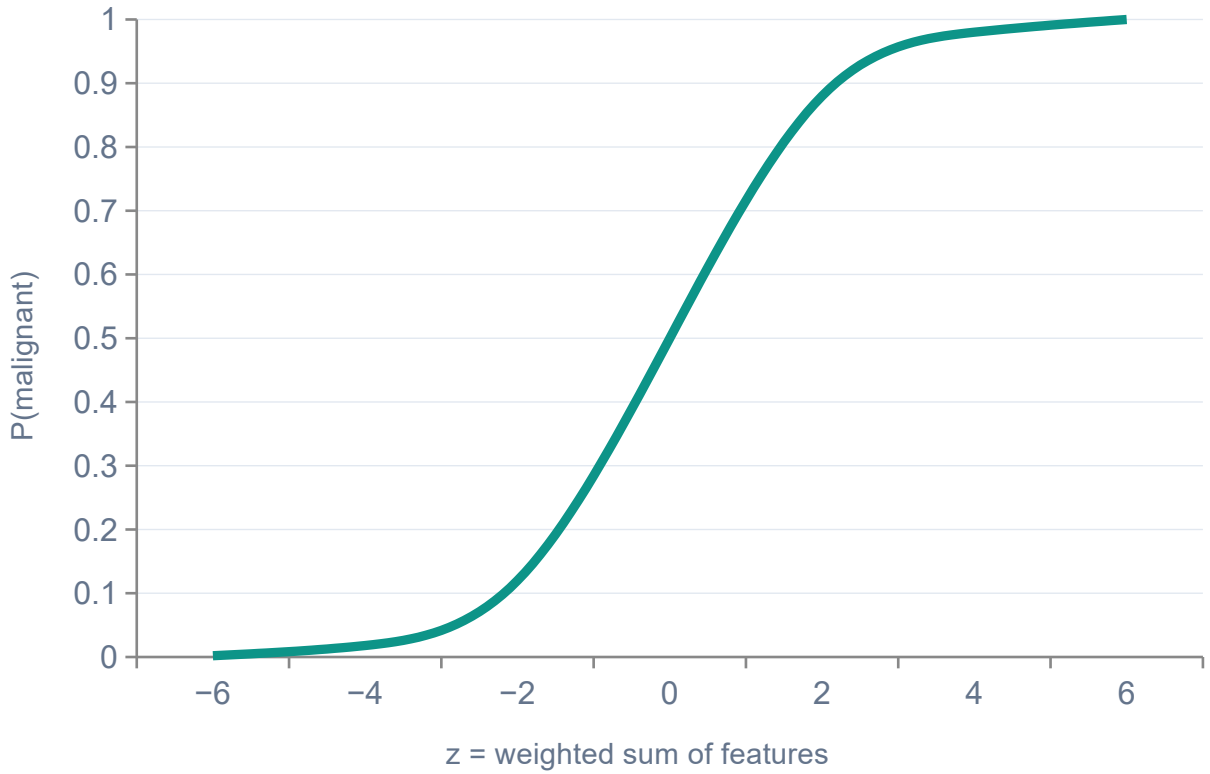
# A classifier draws a boundary




- **Same data, same goal — different geometry: a straight line (logistic) · axis-aligned rectangles (tree) · wiggly neighborhoods (KNN) · a smooth curve (RBF-SVM)**
- Real fits on breast cancer (mean radius × mean texture, standardized) — test accuracy printed on each panel
- **predict() returns the class; predict_proba() returns the probability — every sklearn classifier offers both**

BLACK BOX #2

# Logistic regression: a probability pump




- **z = w·x + b  (same as linear!)**
- sigmoid squashes z into probability 0–1
- **predict_proba → the number**
- predict → threshold at 0.5 by default
- **"Logistic regression" = classification. The name lies.**

```
CV accuracy 0.974 · ROC-AUC 0.995
```

INTERPRETATION

# Logistic weights = evidence, in log-odds

| feature (standardized) | weight | reads as |
|---|---|---|
| worst texture | −1.25 | coarser texture → malignant |
| radius error | −1.07 | size variability → malignant |
| worst symmetry | −0.96 | asymmetry → malignant |
| worst concave points | −0.94 | concavities → malignant |
| area error | −0.94 | area variability → malignant |

(Signs here are data-specific; magnitude = strength of evidence. Positive class = benign in this encoding.)

- **Same reading skill as S05**
- linear: weights move the VALUE
- logistic: weights move the PROBABILITY
- exp(w) = odds ratio — "evidence multiplier"
- **Trains fast, scales big — the industry default first classifier**

BLACK BOX #3

# Decision trees: 20 questions for tumors

**"worst radius ≤ 16.8?"**
no → malignant-ish branch

→

**"concave points ≤ 0.14?"**
yes → likely benign

→

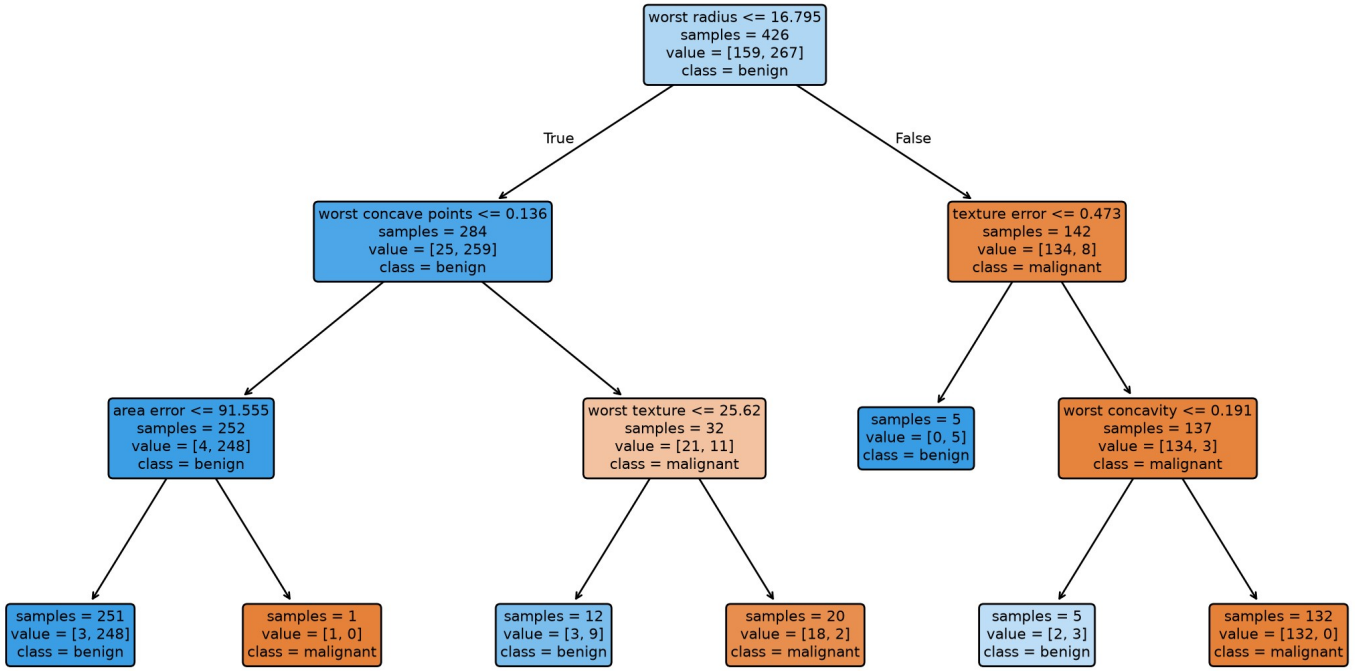
**"area error ≤ 91.6?"**
refine again

→

**leaf**
predict class + proba

- **How is a question chosen?**
- try candidate thresholds per feature
- keep the split that best separates the classes (Gini / entropy — purity math, intuition is enough)
- Stop when: max_depth / min_samples_leaf / pure leaves
- No scaling needed — thresholds compare raw values

# A real fitted tree, read aloud




plot_tree(tree3) — the same depth-3 model, drawn as a flowchart (test acc 0.944)

- **This IS the model**
- export_text(tree, feature_names=…)
- a doctor can audit every decision
- "Small, smooth tumors are benign" — the data says so
  - plot_tree() draws it as a flowchart

# Unleashed, a tree memorizes everything

| tree | train acc | CV acc | AUC |
|---|---|---|---|
| unrestricted | 1.000 | 0.910 | 0.900 |
| max_depth = 3 | 0.977 | 0.924 | 0.934 |

- **A free tree grows until every leaf is pure**
- one patient per leaf → 100% train accuracy
- **its CV accuracy (0.910) is the WORST of today's models**
- restrict depth / min_samples → generalizes better
- **...or grow many restricted trees and average them → RandomForest (soon)**

### Also: trees are unstable

Two random samples → two quite different trees.

High variance is the disease.
Ensembles are the cure.

Remember this slide — the next one pays it off.

# Naive Bayes & SVM

### GaussianNB — 0.930 acc / 0.986 AUC

Assumes features are independent given the class (naive — they never are).

And still works shockingly well.

· trains in a blink, tiny data OK
· THE classic for spam & text (S06 of any NLP course)

Use when: speed, small data, or a strong baseline.

### SVM (RBF) — 0.977 acc / 0.995 AUC ⋆ today's best single

Draws the widest street between classes (max margin), possibly curved via kernels.

· powerful on small/medium clean data
· NEEDS scaling (S03!)
· doesn't scale to millions of rows; no predict_proba by default

Use when: medium data, strong boundaries.

SCOREBOARD 1/2

# Single models, stratified 5-fold CV

| model | CV acc | AUC |
|---|---|---|
| Dummy (most_frequent) | 0.627 | 0.500 |
| DecisionTree (free) | 0.910 | 0.900 |
| GaussianNB | 0.930 | 0.986 |
| KNN (k=5) | 0.963 | 0.985 |
| LogisticRegression | 0.974 | 0.995 |
| SVM (RBF) | 0.977 | 0.995 |

- **Every model beats Dummy by miles — but AUC (0.900 vs 0.995) separates them far better than accuracy, and ranks the free tree LAST**

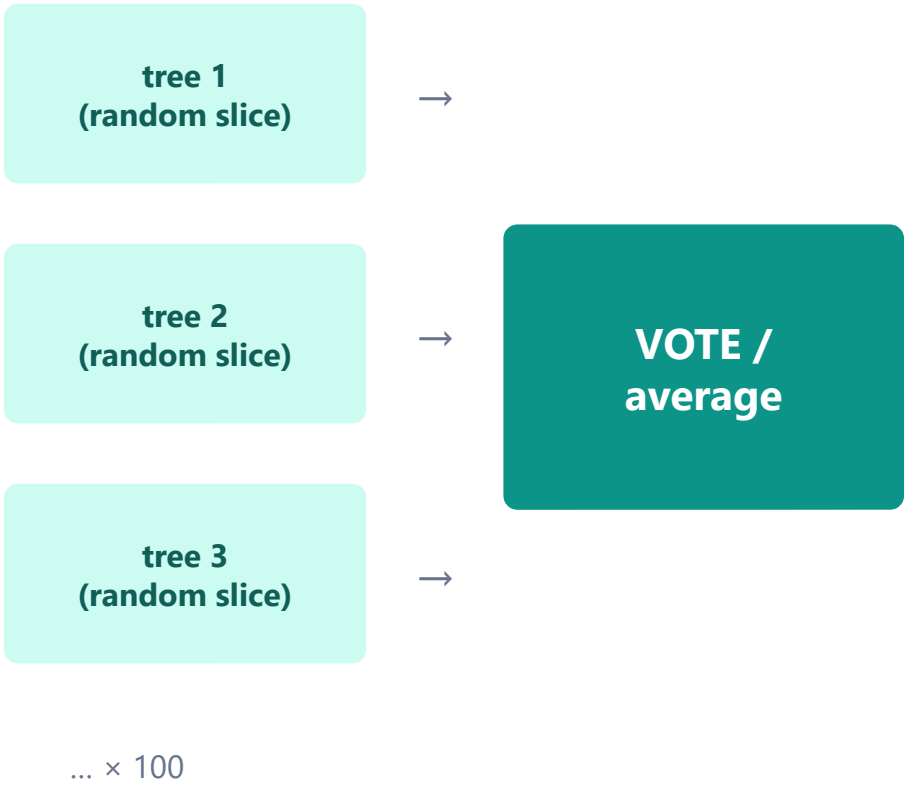
**Next question**

Can we COMBINE models —
take a few imperfect opinions
and beat the best single one?

(The answer is yes —
and it's how real tabular ML
competitions are won.)

# Wisdom of crowds: why committees work

- **One tree: high variance**
- new sample → new tree → new opinions
- **Ask 100 slightly-different trees**
- each sees a random slice of rows & features
- errors are UNCORRELATED-ish → they cancel in the vote
- **Conditions (the ML version of Galton's ox):**
  - each member better than chance
  - members disagree with each other
  - aggregate by voting (classify) or averaging (regress)



# RandomForest: bagging + a random twist

**bootstrap rows**
sample WITH replacement
→
**random features per split**
the "forest" twist
→
**grow deep tree**
no pruning — variance is fine
→
**× 100 trees → vote**
aggregate

| model | CV acc | AUC |
|---|---|---|
| single free tree | 0.910 | 0.900 |
| RandomForest(100) | 0.956 | 0.989 |

- **Feature importances (real, this dataset)**
- worst area 0.15 · concave points 0.13 · mean concave 0.11
- ≈ "which columns did the forest actually use?"
- Caveat: correlated features share/split credit — read as a hint, not a verdict

## Why it works

Deep trees = low bias, high variance. Averaging 100 uncorrelated-ish trees ≈ variance ÷ 100.

Strong default: little tuning, no scaling needed, handles mixed types.

# Boosting: each tree fixes the last one's mistakes

**tree 1**
predicts; notes its misses

→

**tree 2**
focuses on tree 1's misses

→

**tree 3…N**
keeps chasing residuals

→

**sum of all**
shallow but cumulative

| model | CV acc | AUC |
|---|---|---|
| RandomForest(100) | 0.956 | 0.989 |
| HistGradientBoosting | 0.967 | 0.992 |

- **Bagging: parallel, deep trees, vote · Boosting: sequential, shallow trees, add up — HistGB is sklearn's fast boosting (native NaN support!)**
- **Same family beyond sklearn: XGBoost, LightGBM, CatBoost — the kings of tabular Kaggle. Your S08 project can try them.**

# Voting & stacking: mix different species

```
vote = VotingClassifier([
    ("lr",  make_pipeline(StandardScaler(),
                          LogisticRegression(max_iter=5000))),
    ("rf",  RandomForestClassifier(random_state=42)),
    ("knn", make_pipeline(StandardScaler(),
                          KNeighborsClassifier()))],
    voting="soft")   # average PROBABILITIES
```

- **hard voting**
- each model votes a class label
- **soft voting (better)**
- average predict_proba — confidence counts
  - Stacking: a small "meta-model" learns to weight the members — sklearn's StackingClassifier
- **Members should DISAGREE (diversity > quantity)**

| model | CV acc | AUC |
|---|---|---|
| Voting(LR + RF + KNN) | 0.972 | 0.995 |

SCOREBOARD 2/2

# The full table — and its three lessons

| model | CV acc | AUC |
|---|---|---|
| Dummy (most_frequent) | 0.627 | 0.500 |
| DecisionTree (free) | 0.910 | 0.900 |
| GaussianNB | 0.930 | 0.986 |
| RandomForest(100) | 0.956 | 0.989 |
| KNN (k=5) | 0.963 | 0.985 |
| HistGradientBoosting | 0.967 | 0.992 |
| Voting(LR+RF+KNN) | 0.972 | 0.995 |
| LogisticRegression | 0.974 | 0.995 |
| SVM (RBF) | 0.977 | 0.995 |

- **Lesson 1**
- ensembles lifted the tree 0.910 → 0.956+
- **Lesson 2**
- on this easy set, tuned singles match ensembles —
- no free lunch; ALWAYS run the table (S04 method)
- **Lesson 3**
- on bigger, messier tabular data, boosting usually wins
- (that's why S08 lets you try XGBoost)

GUIDE

# Which classifier when?

| situation | start with | strength | watch out |
|---|---|---|---|
| Baseline / speed / text | GaussianNB | instant | independence lie |
| Explanation: audit trail | Logistic / small tree | readable | linear-ish |
| Small-medium, clean | SVM (RBF) | top accuracy | scaling + scale-up |
| Tabular default | RandomForest | robust, no scaling | big memory |
| Max tabular accuracy | HistGB / XGBoost | Kaggle kings | tuning (S07) |
| squeeze out the last % | Voting / Stacking | diversity pays | complexity |

And remember: the winner varies per dataset — the S04 comparison protocol is the real skill.

LAB 06 · DO IT IN GOOGLE COLAB

# ประกวดโมเดลจัดหมวดมะเร็งเต้านม — รวมถึง **ensembles**

**YOUR MISSION (โจทย์ภาษาไทยใน Colab)**

- เทียบ Dummy / LogisticRegression / GaussianNB / SVM / KNN / DecisionTree ด้วย StratifiedKFold (acc + ROC-AUC)
- โชว์กฎของ DecisionTree(max_depth=3) ด้วย export_text แล้วอ่านออกเสียงเป็นประโยคไทย
- พิสูจน์ ensemble effect: tree เดี่ยว vs RandomForest vs HistGradientBoosting
- ดู feature_importances_ ของ RandomForest — ฟีเจอร์ไหนสำคัญที่สุด ตรงกับความรู้ทางการแพทย์ไหม
- โจทย์ชวนคิด: สร้าง VotingClassifier ของตัวเอง (เลือกสมาชิกให้หลากหลาย) แล้วดูว่าชนะสมาชิกตัวไหนไหม

## Learning goal

Run today's full zoo on breast cancer with stratified CV, read real trees and importances, and prove the ensemble effect yourself.

## Vibe coding corner

ลองถาม AI: “อธิบายความต่างระหว่าง bagging กับ boosting เหมือนสอนเด็ก 12 ขวบ” แล้วเขียนคำอธิบาย 3 บรรทัดเป็นภาษาไทยใน markdown cell — เช็กว่าตรงกับสไลด์ไหม

## KEY TAKEAWAYS

1. Classifiers differ in the boundaries they draw: lines (logistic/SVM), rectangles (trees), neighborhoods (KNN).
2. Logistic turns evidence into probability; small trees turn it into rules you can audit.
3. A free tree memorizes (CV 0.910, worst of the day) — restriction or aggregation fixes it.
4. Bagging averages deep trees (RandomForest 0.956); boosting chains shallow ones (HistGB 0.967); voting mixes species (0.972).
5. No universal winner — the S04 protocol (same folds, right metric, Dummy anchor) is the real skill.

## NEXT

Session 07 — Unsupervised Learning + Hyperparameter Tuning — structure, and squeezing